

Below is a detailed overview of ARM architecture essentials and how C code typically maps to ARM assembly. I'll cover core architectural features, common instructions, calling conventions, and some typical examples. This should help you prepare for exam questions related to writing or interpreting assembly on ARM and understanding how C translates to assembly.

1. ARM Architecture Overview

1.1. RISC Principles

- **RISC (Reduced Instruction Set Computing):** The ARM architecture follows the RISC design principles, meaning:
 - A relatively small set of instructions (though ARM has grown more complex over time).
 - Most instructions execute in a single cycle (on simpler pipeline designs).
 - Load/store architecture (memory is accessed only through specific load and store instructions).

1.2. Registers and Program Status

- **General-Purpose Registers (GPRs):** ARM typically has 16 integer registers (R0–R15):
 1. **R0–R3:** Used for function parameter passing and scratch registers.
 2. **R4–R11:** Callee-saved registers (i.e., a function must preserve their values if it changes them).
 3. **R12 (IP):** Intra-Procedure-call scratch register (sometimes used by compilers in prologue/epilogue).
 4. **R13 (SP):** Stack pointer.
 5. **R14 (LR):** Link register (holds the return address on function calls).
 6. **R15 (PC):** Program counter.
- **Program Status Register (CPSR):**
 1. Holds the condition flags: **N** (Negative), **Z** (Zero), **C** (Carry), **V** (Overflow).
 2. Also holds processor mode bits (e.g., user mode, supervisor mode), interrupt disable flags, and other status bits.

1.3. Endianness

- ARM can be either little-endian or big-endian, but the most common configuration (especially in general-purpose processors like ARM Cortex series) is little-endian.

1.4. Pipeline and Instruction Execution

- **Pipeline:** Classic ARM processors often have a 3-stage pipeline (fetch, decode, execute). Modern ARM cores can have deeper pipelines (Cortex-A series).
- **Conditional Execution:** ARM instructions can be conditionally executed based on the condition flags in the CPSR. For example, `ADDEQ` will only execute if the zero flag (Z) is set.

2. ARM Instruction Set Basics

2.1. Instruction Categories

1. **Data Processing:** Arithmetic and logical (e.g., ADD, SUB, AND, ORR, EOR, MOV, MVN, CMP, etc.).
2. **Load/Store:** Move data between memory and registers (e.g., LDR, STR, LDM, STM).
3. **Branch/Control Flow:** Jump to other instructions (e.g., B, BL, BX).
4. **Multiply and Multiply-Accumulate:** (e.g., MUL, MLA).
5. **Synchronization/Advanced:** For multiprocessor systems or more advanced usage (e.g., SWP, LDREX, STREX in modern ARM).

2.2. Condition Codes

- Most ARM instructions allow an optional condition suffix:
 - EQ (Z=1), NE (Z=0), CS/HS (C=1), CC/LO (C=0), MI (N=1), PL (N=0), VS (V=1), VC (V=0), GE, LT, GT, LE, AL (always execute).
- Example: `ADDNE R0, R0, #1` will add 1 to R0 only if the Z flag is 0.

2.3. Addressing Modes for Load/Store

- **Immediate offset:** `LDR R0, [R1, #4]`
- **Register offset:** `LDR R0, [R1, R2]`
- **Pre-indexed:** `LDR R0, [R1, #4]!` (updates R1 with R1+4 after loading)
- **Post-indexed:** `LDR R0, [R1], #4` (use R1's old value for load, then increments R1 by 4)

2.4. Typical Data Processing Examples

- `MOV R0, #5` → $R0 = 5$
- `ADD R1, R2, R3` → $R1 = R2 + R3$
- `SUB R2, R2, #1` → $R2 = R2 - 1$
- `CMP R0, #0` → Compare R0 with 0 (sets flags, no register changes)
- `BNE somewhere` → If not equal (Z=0), branch to somewhere

3. Function Calls and the ARM Calling Convention (AAPCS)

3.1. Register Usage in Function Calls

1. **Parameter Passing:** The first four parameters to a function are passed in R0–R3. Further parameters are passed on the stack.

2. **Return Value:** A function's return value is placed in R0 (and R1 for 64-bit returns, depending on the ABI).
3. **Callee-Saved Registers:** R4–R11 must be preserved by the callee if used. R0–R3, R12, and LR can be overwritten freely by the callee.

3.2. Typical Function Prologue/Epilogue

When a function begins (the prologue), it might:

1. Push LR (link register) onto the stack to preserve the return address.
2. Push any callee-saved registers that it plans to use.
3. Adjust the stack pointer to allocate local variables.

When a function is about to return (the epilogue), it:

1. Restores callee-saved registers from the stack (if any were pushed).
2. Pops LR back into R14 (sometimes done with a single instruction like `pop {r4-r7, pc}`, which will also jump to the return address).

A typical small function's ARM assembly might look like:

```
1. myFunction:
2.     PUSH    {R4, LR}          ; Save R4 and LR
3.     ; Possibly move arguments from R0-R3 to local regs if needed
4.     MOV     R4, R0            ; Example usage: store parameter (in R0) into R4
5.     ; ... function body ...
6.
7.     ; function return
8.     MOV     R0, #0            ; Example: return 0
9.     POP     {R4, PC}         ; Restore R4 and return using LR popped into PC
10.
```

4. ARM Assembly with C Examples

4.1. Simple Arithmetic in C

```
1. int add_two_numbers(int a, int b){
2.     return a + b;
3. }
4.
```

Potential ARM Assembly (simplified):

```
1. add_two_numbers:
2.     ADDS    R0, R0, R1      ; R0 = R0 + R1, set flags
3.     BX      LR             ; Return, R0 holds the result
4.
```

- The function's parameters are in R0 and R1.
- The sum is stored back into R0.
- `BX LR` returns (branch to address in LR).

4.2. Using Local Variables and the Stack

```
1. int sum_array(int *arr, int length) {
2.     int sum = 0;
3.     for (int i = 0; i < length; i++) {
4.         sum += arr[i];
5.     }
6.     return sum;
7. }
8.
```

Possible ARM Assembly (annotated, might differ by compiler):

```
1. sum_array:
2.     ; Prologue
3.     PUSH    {R4, R5, LR}      ; Save callee-saved registers and LR
4.     MOV     R4, R0            ; R4 = arr (pointer)
5.     MOV     R5, R1            ; R5 = length
6.     MOV     R0, #0            ; sum = 0 -> R0
7.     MOV     R1, #0            ; i = 0 -> R1
8.
9. loop_check:
10.    CMP     R1, R5             ; compare i with length
11.    BGE     done              ; if i >= length, exit loop
12.
13.    ; sum += arr[i]
14.    LDR     R2, [R4, R1, LSL #2] ; R2 = arr[i], i is in R1, so offset i*4
15.    ADD     R0, R0, R2         ; sum = sum + arr[i]
16.
17.    ; i++
18.    ADD     R1, R1, #1
19.    B       loop_check
20.
21. done:
22.     ; Epilogue
23.     POP     {R4, R5, PC}      ; restore R4, R5, and return via LR
24.
```

- Notice how local variables (`sum`, `i`) are kept in registers (`R0`, `R1`) here rather than on the stack. For more complex functions or a shortage of registers, the compiler may spill them onto the stack.

4.3. Conditionals in C

```
1. int max_of_two(int x, int y) {
2.     if (x > y) {
3.         return x;
4.     } else {
5.         return y;
6.     }
7. }
8.
```

Possible ARM Assembly:

```
1. max_of_two:
2.     CMP     R0, R1            ; Compare x (R0) to y (R1)
3.     MOVGT   R0, R1            ; If x > y, R0 = x (actually might be reversed depending on order)
4.     BX      LR
5.
```

- Alternatively, the compiler may reorder or invert the comparison, but the main idea is that condition codes allow a one-line conditional assignment.

5. Common Exam Topics and Tips

1. **Identifying Instruction Types:** You may be asked what type of instruction something is (e.g., data processing, load/store, branch).
2. **Decoding Addressing Modes:** Understand the difference between `[Rn, #offset]!` (pre-index) and `[Rn], #offset` (post-index).
3. **Endianness:** Know how data is laid out in little-endian vs. big-endian memory.
4. **Pipeline Effects:** Early ARM pipeline had “off-by-8” in PC relative addressing. Modern compilers and assemblers often handle that behind the scenes. Just be aware the PC can be “ahead” of the current instruction.
5. **Function Prologue/Epilogue:** Understand how the stack is used to preserve registers. Be sure you know which registers are callee-saved vs. caller-saved.
6. **Parameter Passing:** R0–R3 for the first four arguments, then the stack for subsequent arguments. Return values go in R0.
7. **Assembly vs. C:** You might need to match a short snippet of C code to a small assembly snippet (or vice versa). Focus on:
 - Which registers hold parameters?
 - Where is the return value?
 - Are local variables held in registers or on the stack?
 - How are loops or conditionals implemented?

5.1. Recognizing Compiler Optimizations

- **Inlined functions:** The compiler might inline small functions, so they don’t appear as separate calls.
- **Register allocation:** The compiler may rearrange variable usage among registers for efficiency.
- **Instruction scheduling:** The compiler might reorder instructions for pipeline efficiency, but logically the code does the same thing.

6. Putting It All Together: Example Walkthrough

Suppose you have an example exam question like:

“Given the following ARM assembly, describe what the function does in C-like pseudocode.”

```
1. foo:
2.     PUSH    {R4, LR}
3.     SUB     SP, SP, #16      ; allocate 16 bytes for locals
4.     MOV     R4, R0          ; R4 = first parameter
5.     STR     R1, [SP, #4]    ; store second parameter at SP+4
6.     LDR     R0, [R4, #0]    ; R0 = *R4
7.     LDR     R1, [SP, #4]    ; R1 = second parameter from stack
```

```
8.    ADD    R0, R0, R1
9.    ADD    SP, SP, #16    ; deallocate local space
10.   POP    {R4, PC}
11.
```

A likely interpretation is:

1. The function takes two parameters:
 - The first in R0, which is a pointer $R4 = R0$.
 - The second in R1, stored on the stack at $[SP, \#4]$.
2. It reads the integer pointed to by R4.
3. Then it adds the second parameter's value to that integer.
4. Returns the result in R0.

So a C version could be:

```
1. int foo(int *ptr, int val) {
2.     return *ptr + val;
3. }
4.
```

7. Study Recommendations

1. **Practice with a Cross-Compiler:** If time permits, you can use something like `arm-none-eabi-gcc` (for embedded) or an ARM Linux cross-compiler. Compile small C snippets with optimization turned off (`-O0`) to produce more readable assembly. Then practice stepping through that assembly.
2. **Learn Common Assembly Patterns:** Recognize prologues and epilogues (pushing/popping registers), the role of `MOV`, `ADD`, `CMP`, `BNE`, etc.
3. **Focus on the Basics:** If your exam includes writing simple assembly functions, get comfortable with:
 - Setting up the stack frame (`PUSH` registers, adjust `SP`).
 - Accessing function parameters in `R0–R3` (and on the stack for additional parameters).
 - Returning values in `R0`.
4. **Work Through Examples:** Write short C functions (e.g., sum of an array, factorial, fibonacci) and look at the compiler-generated assembly.

Final Words

ARM assembly can appear daunting due to its many addressing modes and optional condition codes, but once you understand the fundamental patterns—especially around function calls and memory accesses—you will be well-prepared to handle exam questions that ask for code interpretation or short assembly snippets.